



PROJECT MANIFEST

UWITZ CARDS

CDCDB5A6DB10

PROJECT CODE 002 · AGC002

CREATED DATE 19 SEPTEMBER 2026

THIS MEMORANDUM SETS OUT THE AGREED SCOPE, ARCHITECTURE, SECURITY COMMITMENTS, AND DEVELOPMENT OBLIGATIONS FOR THE UWITZ CARDS NFC BUSINESS CARD PLATFORM, OPERATED UNDER THE AUTHORITY OF UWITZ.

1. Parties & Authority

This document is entered into under the authority of Uwitz. All architectural decisions, security policies, and operational commitments defined herein are binding on all contributors, agents, and automation systems operating within the Uwitz Cards project.

2. Project Summary

Uwitz Cards is a SaaS platform for NFC business cards developed by Uwitz. Users create, manage, and distribute physical NFC cards (NTAG 215) that link to digital profiles – vCards, dynamic redirect links, or corporate team pages.

The platform serves three user classes: individual professionals who want smart NFC cards with contact info and social links; corporate administrators who order cards in bulk, assign them to employees, and manage the organisation's card fleet; and internal super administrators who manage all users, cards, orders, and subscriptions across the platform.

3. Capabilities

3.1 Card Management

- Create, edit, freeze, activate, and convert cards across three types: Social (vCard profile), Corporate (team pages), and TagLink (dynamic redirects)
- Physical NFC chip binding via NTAG 215 chip IDs – the platform links chip → digital profile
- Card tier selection: digital-only or physical (plastic / aluminium pre-order)
- QR code generation in-house via qrcode + Pillow (PNG) and segno (SVG)
- Card image management with upload, storage via Irys, and serving
- Template-driven card creation with structured field mapping

3.2 User Management

- User registration with email/password and bcrypt hashing
- Microsoft Entra ID SSO integration via OIDC
- Role-based access: individual, corporate_admin, admin, logistics_admin
- Session-based authentication with signed cookies (HMAC-SHA256)
- CSRF protection on all state-changing operations
- Referral code generation per user
- Password reset flow via email tokens

3.3 Subscription & Billing

- Stripe integration with three subscription tiers:

- Social Yearly – individual vCard profile
- Corporate Yearly – team management, bulk cards, analytics
- TagLink Monthly – dynamic redirect link
- Stripe webhook receiver for subscription lifecycle events
- Manual subscription management by admins
- Card type conversion with plan mapping (Social ↔ TagLink)

3.4 Order Management

- Physical card ordering with material selection (plastic / aluminium)
- Shipping address collection and order tracking
- Order status workflow managed by logistics admins
- Stripe fee calculation (3% rate, \$0.50 minimum)
- Corporate bulk ordering support

3.5 Corporate Administration

- Organisation management with name, slug, and custom domain settings
- Member invitation and role assignment within organisations
- Team-wide card dashboards and analytics
- Corporate admin dashboard for fleet oversight

3.6 TagLink Dynamic Redirects

- Dynamic redirect URL management per card
- API key creation and management for programmatic URL updates
- Content field with 500-character limit for redirect payloads

3.7 Logistics Administration

- Card inventory tracking and shipping management
- Order detail views for fulfilment workflows
- Logistics admin role with scoped access

3.8 Super Admin Dashboard

- User management: list, create, edit, suspend users
- Full card visibility across all users
- Order management across the platform
- Subscription oversight and manual adjustments
- Pricing configuration
- Orphaned resource detection and cleanup
- Email log viewing

4. Architecture

4.1 System Overview

Browser (Jinja2 + HTMX) Server-rendered templates with HTMX for interactivity. Vanilla CSS dark theme. | | HTTPS (session cookie auth + CSRF) v FastAPI (Python) Authentication, card CRUD, order processing, subscription management, admin operations, webhook handling, image processing, QR generation. | | async driver (motor) v MongoDB All persistent state: users, cards, orders, subscriptions, organisations, card_images, taglink_api_keys, email_logs.

External Services Stripe (payments) · Microsoft Entra ID (SSO) · Irys (media storage) · SMTP2Go (email) · GenQRCode.com (QR codes)

4.2 Components

Component	Role
main.py	FastAPI application entry point: middleware stack, router mounting, startup hooks, health endpoint, landing page
app/auth.py	Authentication layer: password hashing (bcrypt), session management, role-based access guards (individual, corporate_admin, admin, logistics_admin)
app/api/*	REST API routers: auth, cards, users, orders, subscriptions, taglink, images, admin, corporate, webhooks, card_templates, qr
app/dashboard/*	Server-rendered dashboard routes: user dashboard, corporate admin, super admin, logistics – all Jinja2 + HTMX
app/models.py	Pydantic v2 validation models for all request/response schemas
app/config.py	Environment-based settings via pydantic-settings: MongoDB, Stripe, SMTP, Entra ID, Irys credentials
app/database.py	MongoDB connection management via motor (async) with index ensurement
app/email.py	Transactional email via SMTP2Go: welcome, password reset, notifications
app/api/qr.py	In-house QR code generation: PNG via qrcode + Pillow, SVG via segno, content-type encoding (text, WiFi, SMS, email, vCard, crypto, geo), logo overlay, colour customization
app/ card_templates.py	Template registry for structured card creation workflows
vcard_builder.py	vCard 3.0 generation from profile data for NFC card payloads

5. Security Commitments

All systems, communications, and data handling within Uwitz Cards shall meet or exceed the standards defined in this section. These are non-negotiable obligations.

5.1 Authentication & Identity Verification

- All user authentication shall use bcrypt password hashing with per-user salts – plaintext passwords are never stored or transmitted.
- Microsoft Entra ID SSO integration via OIDC with server-side-only client secret – credentials never exposed to the browser.
- Session tokens shall be stored in signed cookies (HMAC-SHA256) with automatic expiry and rotation.
- CSRF tokens shall be required on all state-changing form submissions.
- Auth failures shall return generic errors only – no information leakage regarding account existence or authentication state.
- Role-based access control enforced at both API and dashboard layers with four tiers: individual, corporate_admin, admin, logistics_admin.

5.2 API Security

- All API requests shall be authenticated via Bearer token or session cookie.
- Rate limiting at 100 requests per 60 seconds per IP.
- CORS configured to a single allowed origin – no wildcard origins.
- Input validation via Pydantic v2 models on all endpoints – invalid input rejected at the boundary.

- Stripe webhook payloads shall be validated via signature verification before processing.
- Request ID middleware for full traceability across all endpoints.

5.3 Data-at-Rest & Data-in-Transit

- All network traffic shall use TLS 1.3 minimum.
- MongoDB access restricted to authenticated, authorised connections.
- Secrets (Stripe keys, SMTP credentials, Entra client secrets, session secrets) shall never be committed, printed, logged, or stored in source code.
- Card images stored via Irys with server-side access control – no public URLs without authentication context.
- Sensitive fields (API key hashes, password hashes) never returned in API responses.

5.4 Compliance & Threat Model

- All new features shall undergo security review against OWASP Top 10 before deployment.
- The security priority order is absolute: Security > Privacy > Correctness > Reliability > Maintainability > Performance > Convenience.
- Public-facing pages shall never expose real personal information – generic placeholders only.
- User IDs follow a structured format ({10-digit-random}.{unix_timestamp}) for consistency and auditability.

6. Data Layer

All persistent state is stored in MongoDB, accessed via the motor async driver with Pydantic v2 model validation.

Key data domains:

Identity & Access	Commerce & Cards
Users (credentials, roles, org memberships)	Cards (type, tier, chip_id, vcard_data, redirect_url)
Organisations (name, slug, custom domain)	Orders (type, material, shipping, status)
Sessions (signed cookies)	Subscriptions (plan, status, Stripe IDs)
Entra ID sync mappings	Card images (Irys storage references)
Referral codes	TagLink API keys (hashed)
	Email logs (transactional audit trail)

7. Deployment Model

7.1 Server

- Single Python application deployed via Uvicorn behind a reverse proxy on bare metal or VM
- Configuration via .env file and pydantic-settings with environment variable override
- MongoDB instance for all persistent state
- systemd service (cards.service) for process management and auto-restart
- GitLab CI/CD pipeline with push-to-deploy workflow: develop locally → push to GitLab → pull on server → restart service

7.2 Frontend

- Jinja2 server-rendered templates with HTMX for client-side interactivity
- Vanilla CSS dark theme (no framework) with design tokens:
 - Background: #09090A through #222225
 - Accent: Red #DA2A1C – primary brand color
 - Fonts: Geist Mono (display), Geist (body)
 - Text: Warm off-white #F3F1EF, muted #BDB9B6
- Responsive layout with mobile support
- Static assets served directly by FastAPI

7.3 External Dependencies

Service	Purpose
MongoDB	Primary data store – users, cards, orders, subscriptions, organisations
Stripe	Payment processing, subscription lifecycle, webhook-driven billing events
Microsoft Entra ID	Enterprise SSO via OIDC – bidirectional user/group sync
Irys	Decentralised media storage for card images and uploaded assets
SMTP2Go	Transactional email delivery – welcome, password reset, notifications